Relazione per HeavyPocket

Caroli Sofia - Passini Anna - Ricci Gabriele

20 marzo 2026

Indice

Analisi 3
1.1 Descrizione e requisiti 3
1.2 Modello del dominio 4
Design 5
2.1 Architettura 5
2.2 Design dettagliato 7
2.2.1 Gabriele Ricci 7
2.2.2 Anna Passini 9
2.2.3 Sofia Caroli 13
Sviluppo 15
3.1 Testing automatizzato 15
3.2 Note di sviluppo 16
3.2.1 Gabriele Ricci 16
3.2.2 Anna Passini 17
3.2.3 Sofia Caroli 18
3.3 Supporto e riadattamento di codice generato 19
Commenti finali 20
4.1 Autovalutazione e lavori futuri 20
4.1.1 Gabriele Ricci 20
4.1.2 Anna Passini 20
4.1.3 Sofia Caroli 21
4.2 Difficoltà incontrate e commenti per i docenti 21
Guida utente 23
Esercitazioni di laboratorio 26

Capitolo 1

Analisi

L'applicazione si pone come obiettivo quello di aiutare i suoi utenti a gestire in maniera più efficiente le proprie risorse economiche, fungendo da tramite tra la loro "voglia di spendere" e le responsabilità. Questa risulta fondamentale in quanto una rappresentazione grafica aiuta la più facile comprensione e memorizzazione delle informazioni, quindi lo scopo di quest'ultima risulta doppiamente funzionale.

1.1 Descrizione e requisiti

Il software sarà un sistema gestionale delle finanze personali strutturato e pensato per un qualunque cittadino odierno. L'applicazione permetterà di prendere nota personalmente delle transazioni, sia in ingresso che in uscita, grazie a un'interfaccia intuitiva e minimale. A colpo d'occhio sarà visibile l'andamento economico delle spese dell'utente, con una visualizzazione del totale del bilancio e la media di entrate e uscite per il mese corrente. In un'apposita lista saranno presenti le transazioni collegate al conto e caratterizzate dalla *data di creazione*, da un *TAG,* da una *descrizione* e da un *importo* ben specifico. Abbiamo pensato, inoltre, a una pratica funzionalità di *budget* per impostare una soglia mensile di denaro dedicata alle spese; questa avviserà l'utente nel caso di superamento. L'applicazione, infine, mostrerà alcuni *grafici* volti a rendere comprensibili a colpo d'occhio le *categorie più dispendiose* sia per quanto riguarda le entrate sia per le uscite.

Requisiti funzionali

- HeavyPocket permetterà l'aggiunta, la rimozione e la modifica di transazioni monetarie in entrata e uscita;
- Sarà possibile categorizzazione tramite TAG le varie uscite;
- Si potranno filtrare le transazioni tramite TAG, data ed entrata/uscita;
- Sarà possibile definire un budget massimo di spesa e visualizzare quando viene superato;
- HeavyPocket mostrerà le medie mensili e appositi grafici delle categorie più dispendiose;

Requisiti non funzionali

- L'interfaccia dovrà essere semplice, ma comprensibile;
- I grafici dovranno semplificare la visualizzazione delle informazioni;
- L'applicazione funzionerà offline.

1.2 Modello del dominio

HeavyPocket avrà un *conto principale* dato dalla somma delle transazioni. I trasferimenti di denaro saranno contrassegnati con una descrizione, una data e un importo con valore positivo (per le *entrate*) o negativo (per le *uscite*) e da un *tag* ciascuno. Sarà tenuta traccia dello storico delle varie transazioni in un apposito *registro*. HeavyPocket avrà un *budget* impostabile di spesa mensile, ovvero una soglia massima di denaro spendibile che avviserà in caso di superamento. Saranno, inoltre, presenti dei *grafici* che mostreranno le uscite e le entrate divise tra loro e raggruppate per tag. Gli elementi costitutivi del problema sono sintetizzati in *Figura 1.1*.

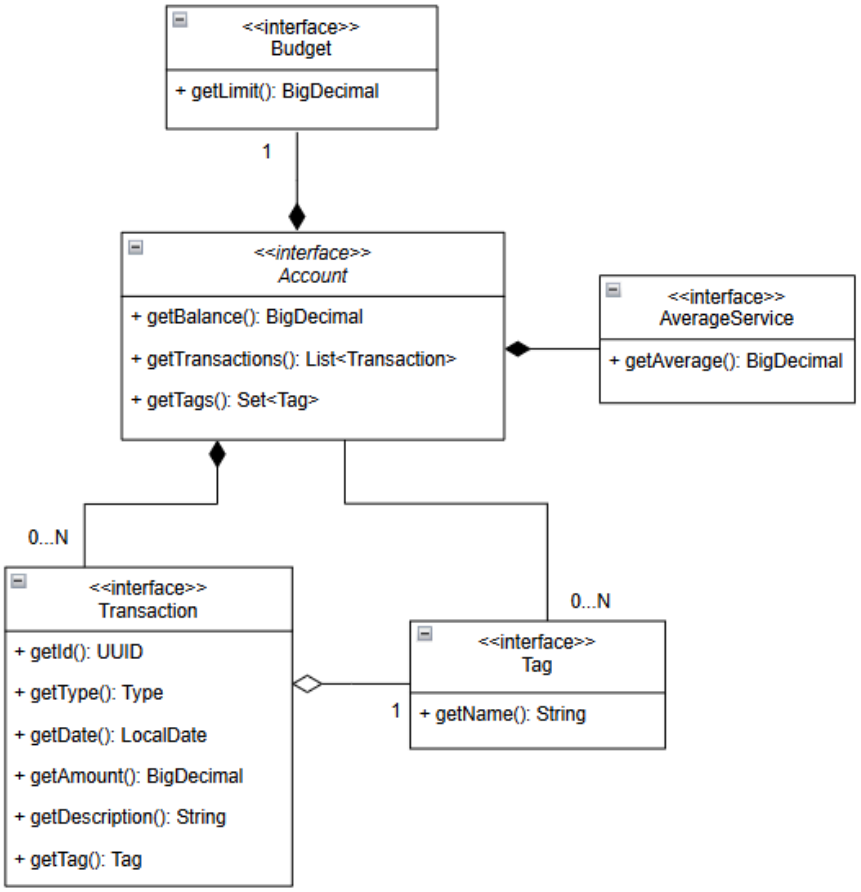

Figura 1.1: Schema UML dell'analisi del dominio con le entità principali e i rapporti fra loro.

Capitolo 2

Design

2.1 Architettura

L'architettura di HeavyPocket segue il pattern architetturale MVC, che prevede una divisione tra logica di business, interfaccia utente e flusso di controllo delle varie entità. Questo ci ha permesso di avere una chiara separazione delle responsabilità e facilitare lo sviluppo dell'applicazione e di un'interfaccia utente ben strutturata. Di seguito è rappresentata una descrizione accurata dei suoi 3 componenti centrali, che cosa contengono e relativi punti di ingresso:

- **Model** è il "cervello" che gestisce i dati e le logiche di calcolo dell'applicazione. Prevede diverse entità fondamentali quali: *Account, Transaction, TransactionBuilder, TransactionType, Tag, Budget e Statistics:*
    - *Account* rappresenta il punto principale dove confluiscono tutte le informazioni, tra cui bilancio totale, operazioni CRUD e di ricerca;
    - *TransactionBuilder* permette la corretta creazione di un oggetto di tipo *Transaction;*
    - *TransactionType* è un enum che permette di classificare le *Transaction* in base alla tipologia della stessa (entrata/uscita);
    - *Tag* è un enum che permette di classificare le *Transaction* in base ad una specifica categoria (food, transport, entertainment, ...);
    - *Budget* permette l'impostazione di un limite da non superare per quanto riguarda le spese mensili;
    - *Statistics* calcola la media di entrate/uscite mensili e fornisce i metodi per inserire nei vari grafici una divisione delle stesse in tag.
- **Controller** è l'intermediario tra Model e View, che coordina entrambi in base a input e azioni ben specifiche:
    - *AccountController* implementa tutti i metodi necessari per permettere la gestione sia del cuore dell'app che delle utility che lo circondano.

- **View** è l'interfaccia grafica che rappresenta i dati e permette l'interazione diretta con l'utente:
    - *Panel* è la radice di tutti i pannelli, che sono stati progettati separatamente per permettere un layout fluido dell'interfaccia;
    - *AccountView* gestisce tutti i vari pannelli raccogliendo gli input e interfacciandosi con il Controller per l'aggiornamento della grafica.

I punti di ingresso dei componenti MVC sono i seguenti:

- nel MODEL è l'interfaccia *Account* (implementata da *AccountImpl*), che offre le operazioni principali operabili sulle *Transaction,* come le CRUD e le ricerche;
- nel CONTROLLER è l'interfaccia *AccountController* (implementata da *AccountControllerImpl*), che funge da filtro logico per dati e operazioni appartenenti agli altri due livelli;
- nella VIEW è *AccountView* (implementata da *AccountViewImpl*), che permette l'avvio grafico dell'applicazione grazie all'installazione dei vari pannelli che costruiscono la GUI. Centralizza poi le operazioni di raccolta input facendo da intermediario con il Controller;

Infine, il progetto espone *HeavyPocketApp*, l'entry point per l'applicativo; questa classe raccoglie e lancia i vari punti di accesso ai tre layer MVC. Il diagramma UML architetturale è rappresentato in *Figura 2.1*.

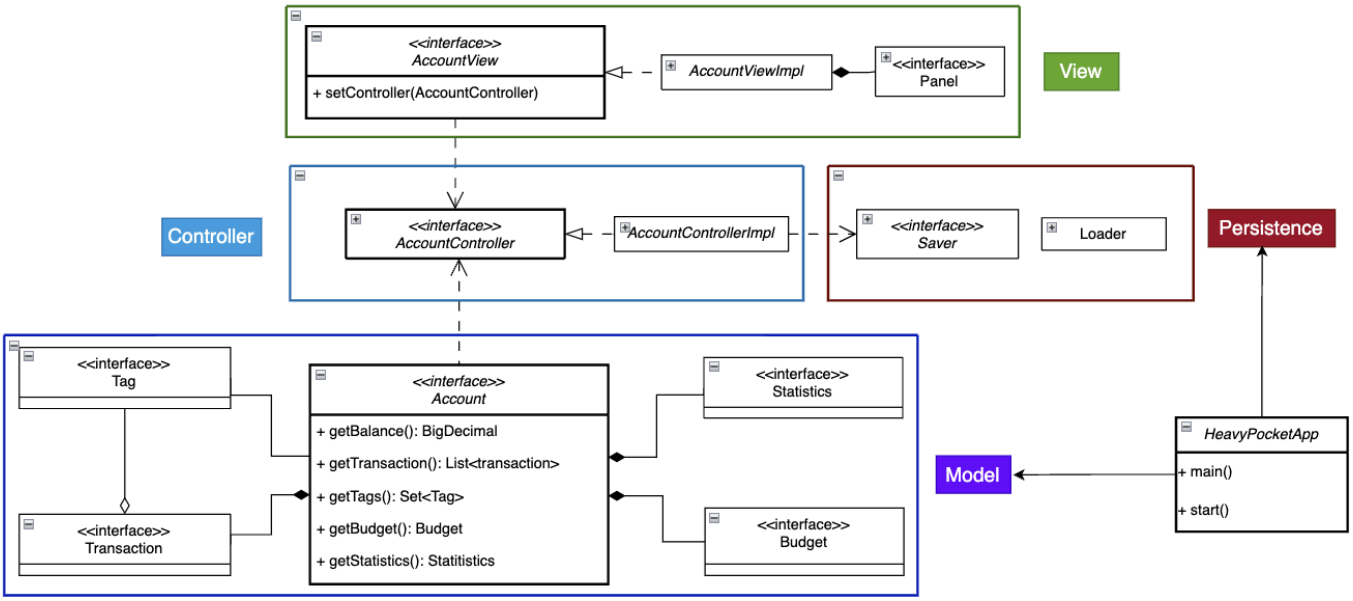

Figura 2.1: Schema UML dell'architettura dell'applicazione.

2.2 Design dettagliato

2.2.1 Gabriele Ricci

Costruzione di oggetti complessi

Problema: la principale entità del progetto è *Transaction*, una classe complessa che richiede molti parametri eterogenei tra loro. Il rischio è che l'istanziazione di questi oggetti sia frequentemente soggetta a bug di creazione a causa della scarsa leggibilità del codice.

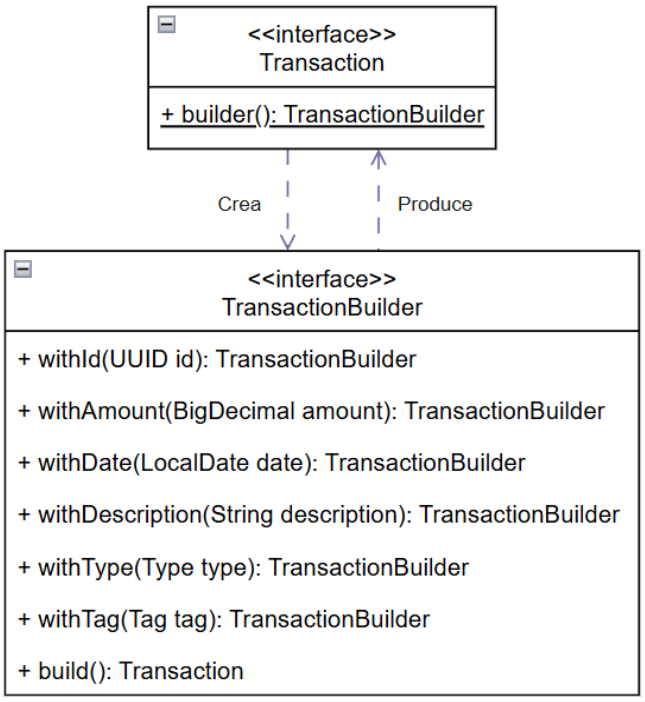

Figura 2.2: rappresentazione UML del Pattern Builder per la creazione di transazioni.

Soluzione: è stato deciso di implementare il *Pattern Builder*. Questa soluzione permette di costruire oggetti complessi in modo ordinato e leggibile: ogni campo della classe non è popolato tramite il passaggio del valore posizionalmente, ma con l'apposito metodo del Builder. In un sistema finanziario, scambiare l'ordine di due parametri è un errore fatale; il builder trasforma il costruttore in una sequenza leggibile e rende l'istanziazione meno equivocabile nel passaggio dei dati. La sua struttura è esemplificata in *Figura 2.2*.

Disaccoppiamento tra View e Model e validazione

Problema: per rendere l'architettura del progetto facilmente aggiornabile nel tempo è necessaria una netta separazione dei layer MVC. I dati partono dal Model e raggiungono la View tramite il Controller; passare direttamente oggetti di dominio alla UI aumenterebbe l'accoppiamento tra i livelli e renderebbe difficile la validazione e gestione di input/output.

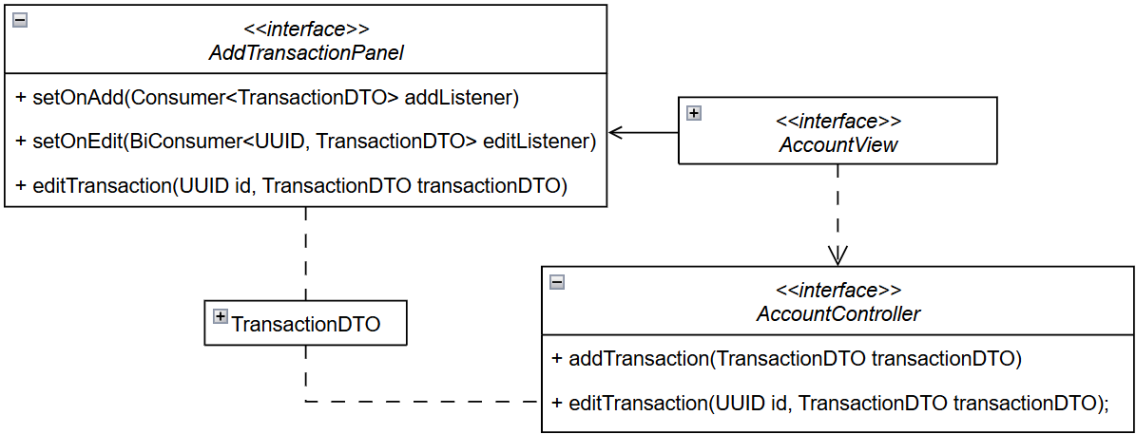

Figura 2.3: Rappresentazione UML del Pattern DTO per il passaggio dati tra View e Controller.

Soluzione: per ridurre l'accoppiamento dei layer è stato implementato il *Pattern DTO* (Data Transfer Object) per i flussi di dati in ingresso dalla UI. In particolare, View e Controller si passano *DTO* evitando di trattare direttamente stringhe sparse o logica di parsing. In questo modo la validazione applicativa e il parsing input sono centralizzati nel Controller che protegge il Model da dati grezzi e dai formati specifici della UI. L'utilizzo semplificato del pattern è visibile in *Figura 2.3*.

Controller orchestratore e architettura Event-Driven

Problema: se i componenti della View invocano direttamente i metodi del Controller, quest'ultimo rischia di diventare un layer passivo; la logica di business sarebbe trasportata nei componenti grafici. Questo creerebbe un forte accoppiamento tra i layer superiori del progetto in quanto i pannelli dovrebbero conoscere l'implementazione sottostante per invocarne i metodi.

```
transactionListPanel.setOnSearch(controller::search);
transactionListPanel.setOnDelete(controller::deleteTransaction);
transactionListPanel.setOnEdit(controller::callToEditTransaction);
addTransactionPanel.setOnAdd(controller::addTransaction);
addTransactionPanel.setOnEdit(controller::editTransaction);
budgetPanel.setOnUpdateLimit(controller::updateBudgetLimit);
```

Figura 2.4: Collegamento Event-Driven tra i pannelli della View e l'AccountController, basato su callback tipizzate e conforme al principio DIP.

Soluzione: è stata implementata la logica *Event-Driven* basata su *DIP* (Dependency Inversion Principle). Invece di avere un pannello che "comanda" il Controller, la UI si limita a notificare eventi tramite interfacce listener definite dalla View stessa ma implementate dal Controller. Questo garantisce che la logica decisionale rimanga centralizzata nel Controller e mantiene i layer isolati e riusabili. La *Figura 2.4* mostra un estratto del codice di *ViewControllerImpl* con il collegamento tra i metodi dei due livelli.

Gestione di componenti add ed edit

Problema: l'applicazione espone la possibilità di modificare le transazioni una volta inserite nella lista grafica; sviluppare form separati per l'inserimento e l'update delle stesse comporterebbe una duplicazione importante di codice UI e logica di validazione. Tale approccio aumenta il rischio di incoerenze nell'interfaccia e rende il lavoro di manutenzione del codice duplicato.

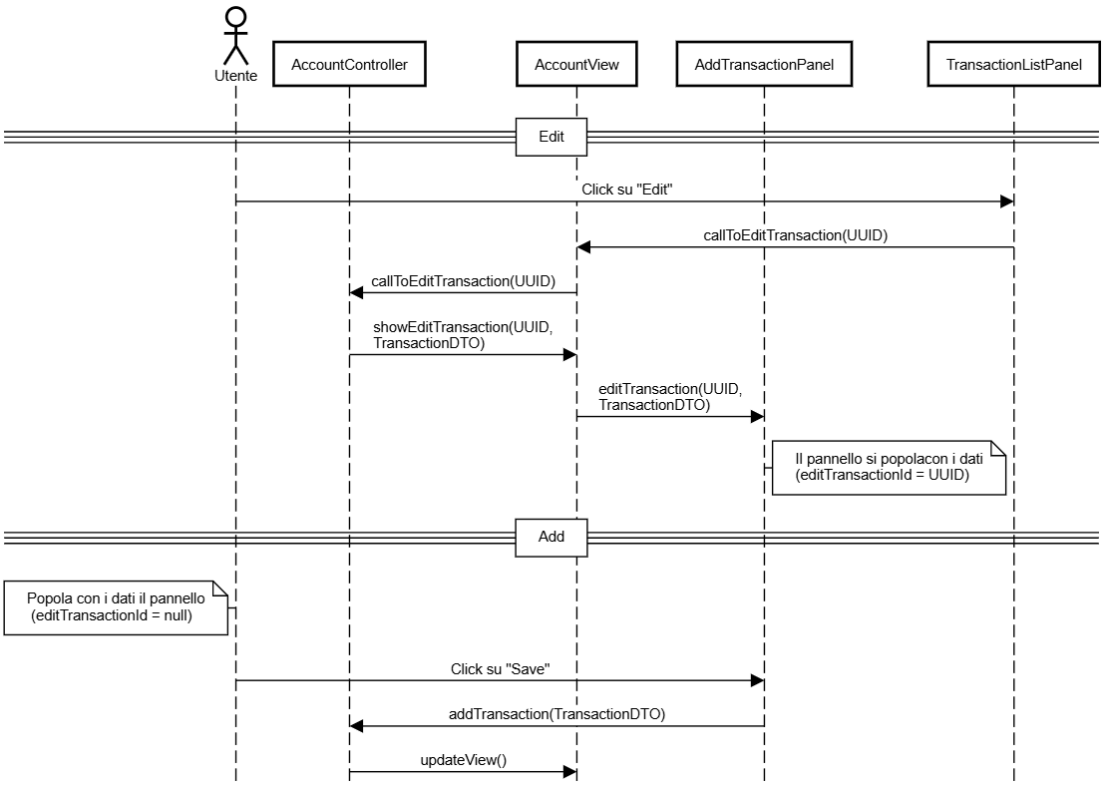

Figura 2.5: Rappresentazione con un diagramma di flusso del riuso del componente AddTransactionPanel.

Soluzione: è stato deciso di gestire l'aggiunta e la modifica delle transazioni in uno stesso componente polimorfico. La logica interna si basa sul campo *editTransactionId*; in base al suo valore il pannello adatta dinamicamente il pre-popolamento dei campi e le proprie callback. Grazie all'uso delle *DTO* in entrambe le implementazioni, il componente garantisce una transizione fluida tra le operazioni di inserimento e modifica, eliminando la duplicazione del codice e assicurando una UI coerente. Il diagramma di flusso in *Figura 2.5* mostra l'andamento delle chiamate.

2.2.2 Anna Passini

Budget Mensile

Problema: il superamento del limite del budget viene calcolato in base al totale delle spese effettuate nel mese corrente presenti sull'account. Bisogna calcolare in modo dinamico la spesa mensile dell'utente, considerando solo le transazioni rilevanti, aggiornando in modo coerente l'interfaccia grafica dopo ogni modifica allo stato del sistema, evitando inconsistenze tra i dati mostrati.

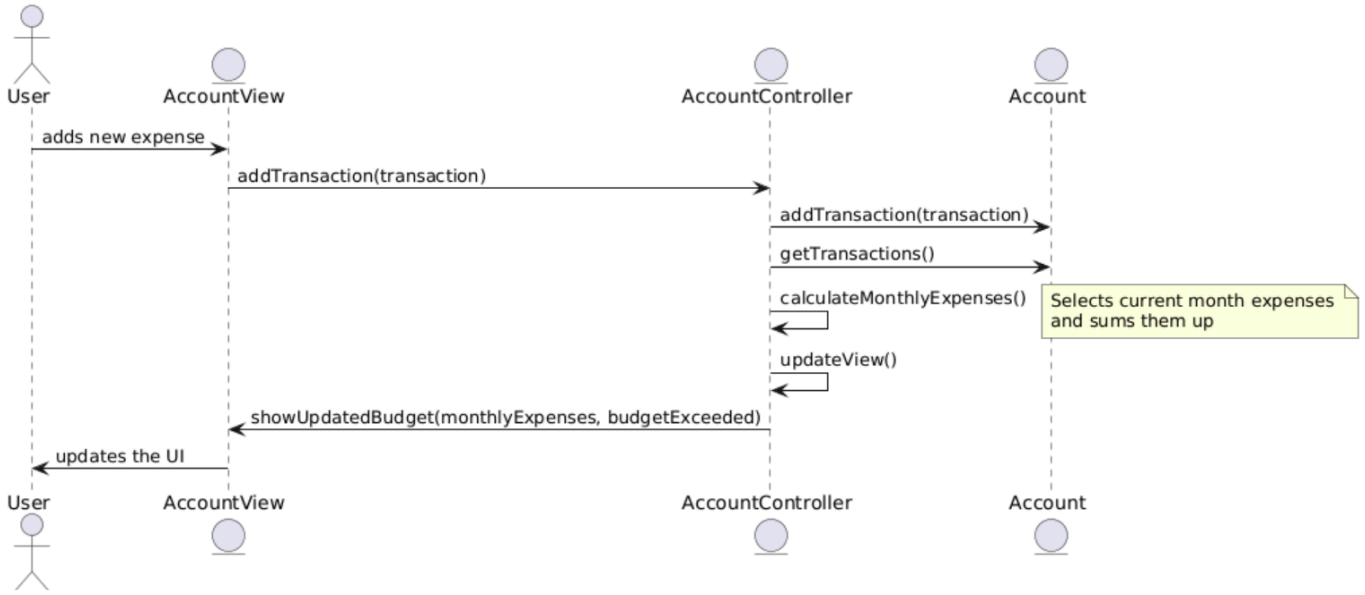

Figura 2.6: Rappresentazione del flusso per l'aggiornamento della UI in base ai cambiamenti rilevati nell'applicazione.

Soluzione: per la raccolta dei dati necessari è stato seguito il flusso rappresentato in *Figura 2.6*: è stato introdotto nel Controller un metodo specifico (*calculateMonthlyExpenses*) che filtra le transazioni del mese corrente e ne aggrega gli importi tramite stream, mentre l'aggiornamento dinamico e automatico della View è stato centralizzato in un unico metodo (*updateView*), che coordina tutte le operazioni di refresh dell'applicazione. Questo approccio consente di isolare la logica di calcolo in un unico punto, migliorandone la leggibilità, la riusabilità e facilitando eventuali modifiche future (ad esempio cambi di criterio temporale), e garantisce che ogni modifica al Model venga riflessa immediatamente e in modo consistente nell'interfaccia.

Problema: era necessario determinare in modo corretto ed efficiente se il budget fosse stato superato, mantenendo separata la logica di controllo dalla rappresentazione grafica del risultato, in accordo con i principi dell'architettura MVC.

Soluzione: è stato implementato il metodo *isBudgetExceeded*, che confronta il totale delle spese mensili con il limite del budget utilizzando *BigDecimal.compareTo*. Il risultato del controllo viene poi comunicato alla View, che si occupa esclusivamente di

mostrarlo all'utente, tramite il metodo *showLimitExceeded*. Questa scelta riflette chiaramente il pattern MVC, in cui il controller gestisce la logica applicativa e la view si occupa esclusivamente della presentazione, evitando qualsiasi accoppiamento diretto con il model.

Persistenza e salvataggio dei dati

Problema: Era necessario gestire il caso in cui il file di persistenza non fosse presente (ad esempio al primo avvio), evitando errori e garantendo comunque un'inizializzazione coerente dell'applicazione.

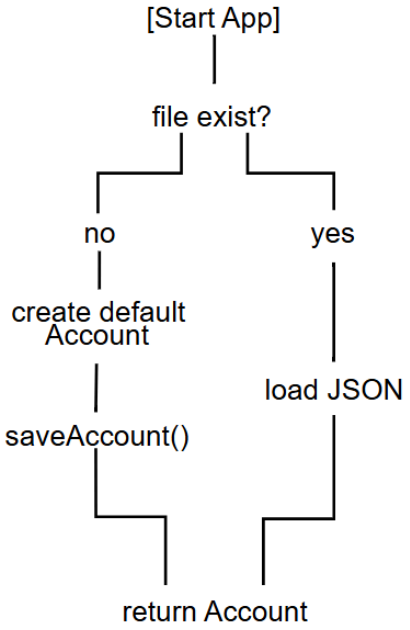

Figura 2.7: Rappresentazione del Flow Logico seguito all'avvio dell'applicazione.

Soluzione: Nel progetto, il metodo statico *loadData* controlla direttamente la presenza del file nel path designato. Come mostrato in *Figura 2.7*, se il file non esiste, crea un Account con valori di default, poi tenta di salvarlo tramite il Saver passato in input. Questa scelta implementa un'inizializzazione *fail-safe*: l'app parte con uno stato valido anche al primo avvio. Se il salvataggio iniziale fallisce viene lanciata *IllegalStateException*; se invece il file esiste ma non è leggibile, il caricamento propaga *UncheckedIOException*.

Problema: Nella progettazione del sistema di persistenza è emersa la necessità di utilizzare un formato di salvataggio che fosse non solo funzionale alla memorizzazione dei dati, ma anche facilmente leggibile, mantenibile ed estendibile nel tempo. L'utilizzo di formati binari o soluzioni proprietarie avrebbe reso più difficile il debugging, la verifica manuale dei dati e l'eventuale evoluzione del sistema, introducendo un forte accoppiamento tra struttura interna del model e rappresentazione su file.

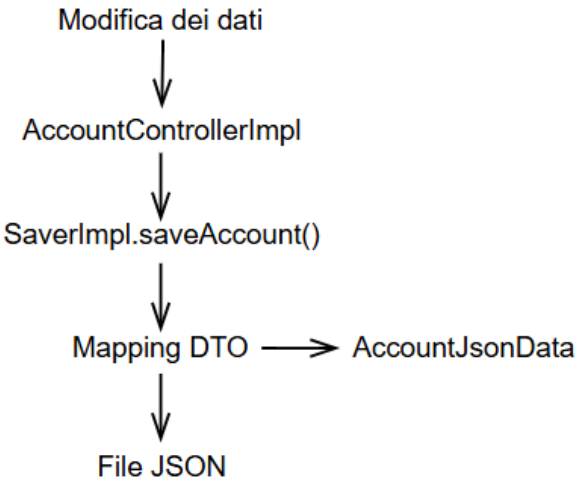

Figura 2.8: Rappresentazione del flusso seguito per il salvataggio dei dati.

Soluzione: Per risolvere questo problema è stato adottato il formato *JSON*, utilizzando la libreria *Gson* per la serializzazione e deserializzazione dei dati. Il *JSON* rappresenta una soluzione standard, leggibile anche dall'utente e facilmente interpretabile durante le fasi di debugging, consentendo di ispezionare direttamente il contenuto del file di persistenza. Inoltre, la sua struttura flessibile permette di estendere facilmente i dati salvati, aggiungendo nuovi campi senza compromettere la compatibilità con versioni precedenti del sistema. L'utilizzo di questo formato è stato integrato con un approccio basato su *Persistence DTO*, in modo da mantenere separata la rappresentazione dei dati dal model applicativo. Questo consente di modificare in futuro il formato di persistenza o la libreria utilizzata senza impattare direttamente sulle classi di dominio, migliorando la manutenibilità e la scalabilità complessiva del sistema. Il flusso generale è rappresentato in *Figura 2.8*.

2.2.3 Sofia Caroli

Disaccoppiamento tra modello dati e visualizzazione grafica.

Problema: il *Pie Chart* è un componente UI che vuole dati in formato *ObservableList<PieChart.Data>*, ma il Model ragiona in termini di *Map<Tag, BigDecimal>* per rappresentare Tag delle transazioni e relativi valori sommati.
Se il pannello grafico conoscesse Tag e BigDecimal, diventerebbe accoppiato al dominio e quindi meno riusabile in futuro perché si adatterebbe a questo contesto specifico.

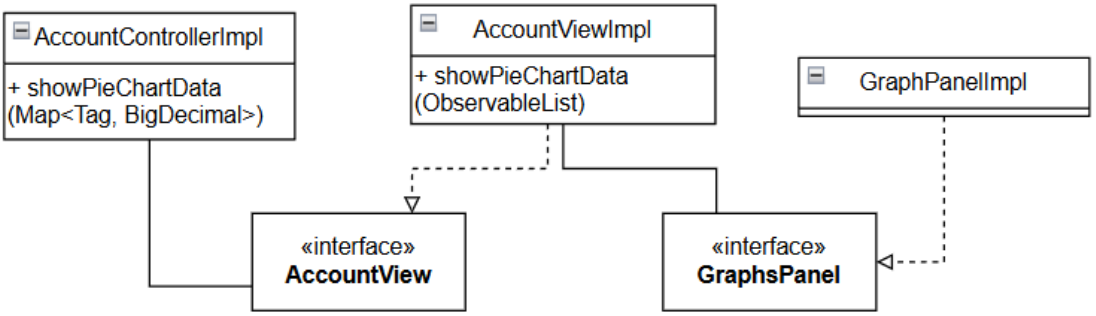

Figura 2.9: Diagramma UML dell'adattamento dei dati per i Pie Chart.

Soluzione: si mantiene *GraphsPanel* come componente puramente legato alla View, che accetta direttamente il formato necessario (*ObservableList<PieChart.Data>*).
La conversione dal dominio (*Map<Tag, BigDecimal>*) al formato corretto viene fatta in *AccountViewImpl*, che è la sezione apposita per adattare un dato a un componente grafico. Il diagramma UML è rappresentato in *Figura 2.9*.

Unificazione della gestione dei grafici di spesa e di entrata.

Problema: la schermata deve mostrare due *Pie Chart* con la stessa logica di presentazione (testo per il titolo e rappresentazione grafica), variando solo il set di dati visualizzato (expense e income). Senza una struttura chiara si rischiano duplicazioni di codice con variazioni minime e incoerenze grafiche nel comportamento dei grafici.

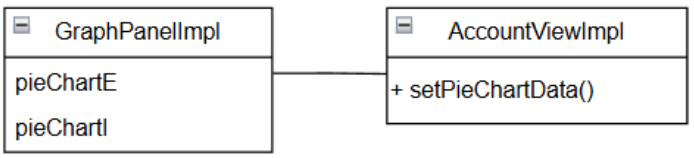

Figura 2.10: Diagramma UML della gestione centralizzata dei due grafici.

Soluzione: è stato scelto di raggruppare i due grafici nello stesso GraphsPanel e offrire un singolo punto di aggiornamento (*setPieChartData()*) in modo da mantenere il layout unificato, permettere al Controller di prepara i dati correttamente e passarli insieme alla View. Il diagramma UML è rappresentato in *Figura 2.10*.

Comporre i pannelli mantenendo la UI flessibile

Problema: la schermata principale è composta da più pannelli (*TransactionListPanel*, *StatisticsBalancePanel*, *AddTransactionPanel*, *BudgetPanel*, *GraphsPanel*). Se *AccountViewImpl* si occupasse anche di creare direttamente le implementazioni concrete (es. *new GraphsPanelImpl()* ecc.), si introdurrebbe accoppiamento tra la View e le classi concrete dei pannelli, difficoltà nel riuso/sostituzione e crescita delle responsabilità della View.

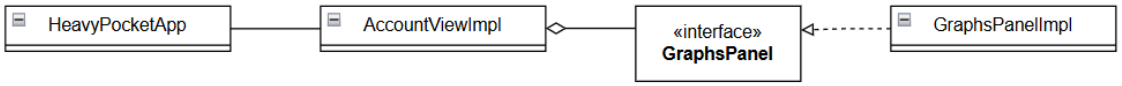

Figura 2.11: Diagramma UML del flusso per la composizione dei pannelli.

Soluzione: *AccountViewImpl* riceve i pannelli già costruiti via costruttore, in questo modo la View si occupa solo della loro composizione, mentre la creazione è delegata al launcher dell'applicazione (*HeavyPocketApp*). Questa soluzione utilizza il design pattern "*Dependency Injection*", poiché i pannelli inseriti nella View (quindi le sue dipendenze) vengono forniti da una fonte esterna. Il diagramma UML è rappresentato in *Figura 2.11*.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il testing è stato realizzato utilizzando la libreria *JUnit*. Ogni partecipante del gruppo ha realizzato dei test automatici relativi alla propria parte del Model. Ci siamo approcciati per la prima volta al TDD cercando di ipotizzare eventuali fallimenti dell'applicazione durante la stesura delle interfacce, ma parte dei test sono stati comunque effettuati test-after. I soggetti dei test sono state le classi:

- *Transaction:* controllo sulla **creazione** delle singole transazioni tramite il *Pattern Builder* e verifica dell'**integrità dei dati**, analisi sulla **correttezza degli importi** restituiti (sia delle entrate che delle uscite) e controllo della robustezza del sistema tramite il **lancio di eccezioni** a fronte di parametri non validi.

- *Budget:* test sull'aggiunta di **valori negativi, null o zero** come valore di *Budget*, controllo sul corretto **salvataggio** del nuovo valore di Budget.

- *Statistics:* il test viene effettuato sulla corretta esecuzione della **media**, sull'individuazione di una **lista vuota** e sulla corretta restituzione di una lista contenente solo le transazioni appartenenti a una **specifica tipologia**.

3.2 Note di sviluppo

3.2.1 Gabriele Ricci

Di seguito un singolo esempio:

- Utilizzo di *lambda expressions* per event handling in JavaFX: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/TransactionListPanelImpl.java#L132-L135

- Utilizzo di *method references* per il collegamento tra View e Controller: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/impl/AccountViewImpl.java#L80

- Utilizzo di *Stream API* per ciclare, filtrare e ottenere dati dalle transazioni e sulla lista di tag: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/controller/impl/AccountControllerImpl.java#L63-L65

- Utilizzo di *Optional* per il filtro delle transazioni per id: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/model/impl/AccountImpl.java#L79

- Utilizzo di *ifPresentOrElse* in conseguenza al tipo di ritorno Optional: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/controller/impl/AccountControllerImpl.java#L104-L114

- Utilizzo di *componenti JavaFx* per la creazione dell'interfaccia: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/AddTransactionPanelImpl.java#L33-L43

- Utilizzo di *interfacce funzionali del JDK (Consumer e BiConsumer)*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/AddTransactionPanelImpl.java#L46-L47

3.2.2 Anna Passini

- Utilizzo di stream() + filter() + toList() per filtrare le transazione per il mese corrente
  https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/controller/impl/AccountControllerImpl.java#L222-L228

- Utilizzo di Bottoni Event-Driven setOnAction() con lambda in BudgetPanel
  https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/BudgetPanelImpl.java#L32-L39

- Utilizzo di Method References Loader::createTransaction
  https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/persistence/impl/Loader.java#L103

- Utilizzo di Interfacce Funzionali JDK Consumer<String> in BudgetPanelImpl
  https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/BudgetPanelImpl.java#L24

- Utilizzo di librerie di JavaFX
  https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/BudgetPanelImpl.java#L3-L9

- Utilizzo di librerie di Gson
  - new Gson() in Loader:
    https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/persistence/impl/Loader.java#L96

  - fromJson() in Loader:
    https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/persistence/impl/Loader.java#L98

  - toJson() in SaverImpl:
    https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/persistence/impl/SaverImpl.java#L51

3.2.3 Sofia Caroli

Di seguito un singolo esempio delle seguenti funzionalità:

- Utilizzo di *Stream API*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/model/impl/StatisticsImpl.java#L19-L29

- Utilizzo di *Method Reference*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/model/impl/AccountImpl.java#L86-L90

- Utilizzo di *Lambda Expressions*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/model/impl/StatisticsImpl.java#L43-L55

- Utilizzo di *Collectors*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/impl/AccountViewImpl.java#L135-L148

- Utilizzo di *JavaFX*: https://github.com/sofcaroli2/heavyPocket/blob/fb990707b0dec5d544bc2032f7b1679b16821761/src/main/java/it/unibo/heavypocket/mvc/view/panels/impl/GraphsPanelImpl.java#L30-L44

3.3 Supporto e riadattamento di codice generato

Per supporto decisionale e correzioni di parti sulle quali ci sentivamo insicuri, abbiamo fatto utilizzo di informazioni ricavate da IA e altri siti o progetti online. Ci teniamo a specificare che l'utilizzo di tali risorse non è stato fonte di copia-incolla, ma solo di ispirazione e rielaborazione. Inoltre, ci è stato particolarmente utile l'esempio di progetto presentato durante il corso dal professore (Pokédex), che è stata una base utile per comprendere la struttura da perseguire. Riportiamo sotto alcuni fonti sitografiche utilizzate sia per le decisioni strutturali, sia per la ricerca di codice:

- https://www.geeksforgeeks.org/system-design/mvc-design-pattern/#components-of-the-mvc-design-pattern
- https://italiancoders.it/observer-pattern/
- https://stackoverflow.com/questions
- https://www.baeldung.com/java-dto-pattern
- https://italiancoders.it/builder-pattern-un-ottima-alternativa-al-costruttore/
- https://docs.oracle.com/javase/8/javafx/api/javafx/scene/chart/PieChart.html
- https://docs.oracle.com/javafx/2/charts/pie-chart.htm
- https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Gabriele Ricci

Durante la stesura del progetto ho avuto diverse difficoltà, soprattutto nella comprensione della struttura architetturale corretta che rispettasse i principi citati a lezione. In più momenti mi sono sentito sconfortato e incerto sulle scelte tecniche fatte e, ancora oggi, non posso dire con sicurezza che il progetto segua la giusta linea implementativa. Per riuscire ad andare avanti ho fatto ampio e necessario uso di fonti esterne, che mi hanno aiutato a prendere le decisioni strutturali. Ho cercato di essere una "figura di riferimento" per i miei colleghi fornendo orientamento e supporto tecnico nonostante le lacune tecniche e le incertezze cercando di mantenere il morale alto anche quando non ero sicuro della scelta migliore. Se in futuro mi ritroverò a lavorare nuovamente con queste tecnologie mi avvarrò delle conoscenze acquisite durante questo progetto e spero di ottenere un risultato più convincente per me stesso in primis.

4.1.2 Anna Passini

Questo progetto mi ha messo duramente alla prova: non mi sentivo sicura in niente di quello che scrivevo, ho riiniziato da zero parecchie volte, ho posto tante, troppe domande a colleghi di tirocinio, di progetto e a intelligenze artificiali diverse per avere conferma di quello che stavo facendo e farmi dare consigli su come procedere.

L'idea di abbandonare il progetto è stata valutata, ma con l'avvicinarsi della possibilità di laurearsi mi sento di aver spinto e convinto il gruppo a provarci comunque e di lavorare un po' di più dopo diversi problemi che abbiamo incontrato per riuscire a consegnare comunque il progetto. Per riuscire a fare tutto ho dovuto rinunciare più volte ad allenamenti e tempo libero, solitamente normale per portare a termine i miei doveri in ogni ambito, ma non era mai successo così ripetutamente in un lasso di tempo ridotto. Queste mancanze (necessarie) mi hanno stressato ulteriormente e hanno sicuramente influito sulla mia resa complessiva.

Le parti che ho prodotto funzionano, ma mentirei affermando di aver capito al 100% tutto e di essere completamente soddisfatta di quello che ho creato; inoltre ho dovuto abbandonare le funzionalità aggiuntive facoltative che inizialmente mi sembravano molto interessanti e mi sarebbe piaciuto implementare, ma, oltre alla mancanza di tempo, la poca fiducia nelle mie capacità che ho maturato durante questo periodo di progettazione mi ha portato ad abbandonare l'idea e concentrarmi di più sulle funzionalità obbligatorie e minime per passare l'esame. Nonostante ciò, posso ritenermi fiera di me stessa in quanto non mi sono lasciata sopraffare e sono riuscita a completare la maggior parte delle cose che mi ero prefissata di fare entro i limiti concessi.

4.1.3 Sofia Caroli

Lo svolgimento di questo progetto è stato per me il primo e vero approccio alla programmazione ad oggetti, quindi sin dall'inizio mi aspettavo di riscontrare non poche difficoltà. La grande quantità di informazioni e concetti che devono essere memorizzati per svolgere al meglio un lavoro in questo ambito, unita alla grande mole di regole da ricordare, mi ha inizialmente destabilizzato abbastanza.

Nonostante questo, l'idea di svolgere il tutto in gruppo mi ha dato il coraggio che mi ha permesso di continuare. Con il proseguire dei giorni riuscivo a immedesimarmi sempre di più nel meccanismo di programmazione e questo mi ha permesso di svolgere un lavoro di cui nel complesso sono soddisfatta.

L'utilizzo delle slide presentate a lezione e di fonti esterne (quali siti, documentazione, ...) mi ha permesso di avere un riscontro immediato su quanto stavo svolgendo. Ritengo che l'idea iniziale, data la vicinanza all'utilizzo comune nella vita, mi abbia aiutato a concretizzare e rendere più comprensibile la realizzazione di certi concetti (transazioni, categorizzazione, budget, ...).

C'è stata confusione iniziale in merito alla struttura da seguire, ma siamo comunque riusciti a portare a termine tutto nei tempi prestabiliti, facendo le opportune rinunce per quanto riguarda lo svolgimento del tirocinio e la prioritizzazione di altri esami.

4.2 Difficoltà incontrate e commenti per i docenti

Il passaggio dalle lezioni allo svolgimento del progetto si è dimostrato molto più complicato di quanto ci aspettassimo e purtroppo ci siamo sentiti come se l'insieme delle competenze fornite non fosse sufficiente a completare i requisiti richiesti, nonostante fossimo studenti del terzo anno e avessimo a disposizione le lezioni per entrambi i semestri trascorsi.

Le conoscenze limitate, l'esperienza mancante e le diverse lacune che avevamo iniziato a maturare, ci hanno portato a richiedere chiarimenti sul fatto e sul da farsi. Nonostante la richiesta di conferma, ai professori, sulla struttura che stavamo perseguendo, la risposta che ci è stata fornita è stata percepita come poco obiettiva e sviante. Questo ci ha portato a rielaborare l'idea iniziale, perseguendo così una struttura errata che ci ha poi portato in corso d'opera a ripartire completamente da zero a ridosso della deadline, sapendo che è penalizzata la consegna in ritardo in qualsiasi evenienza.

A causa della scarsità di tempo abbiamo dovuto rinunciare a presenziare al tirocinio per dedicare maggior tempo al progetto; ci teniamo a sottolineare che questa sovrapposizione è stata una conseguenza aggravata derivata dal fatto che già nella sessione dell'anno passato (nella quale avremmo dovuto dare questo esame) abbiamo dovuto decidere se dare questo o tutti gli altri della sessione. Il plico di lavoro richiesto era secondo noi troppo pesante per essere conseguito insieme a quello degli altri progetti e questa scelta si è poi propagata per le successive 2 sessioni per i medesimi motivi.

Siamo consapevoli che la data per la scadenza di consegna è stata scelta da noi, ma non essendo abituati a un tipo di richiesta di questo genere, le condizioni createsi hanno portato ad alcune mancanze a livello organizzativo.

In sintesi, l'esperienza non è stata del tutto piacevole, non tanto per la scelta del linguaggio o per la tipologia di esame, quanto più per la mancanza di informazioni sul procedimento da seguire e per la rigidezza dei requisiti da rispettare per svolgere e consegnare l'elaborato.

Appendice A

Guida utente

HeavyPocket si pone come obiettivo quello di aiutare i suoi utenti a gestire in maniera più efficiente le proprie risorse economiche, fungendo da tramite tra la loro "voglia di spendere" e le responsabilità. L'applicazione permetterà di prendere nota personalmente delle transazioni, sia in ingresso che in uscita, e di rendere maggiormente comprensibile il proprio bilancio nel tempo grazie ad un'interfaccia intuitiva e minimale, provvista anche di grafici. Il suo obiettivo è rendere più semplice la gestione delle proprie finanze e fornire una panoramica chiara delle spese effettuate.

Per aprire l'applicazione, basta eseguire il file principale del programma che mostrerà la finestra con tutte le funzionalità. L'interfaccia contiene (in ordine di apparizione):

- il *saldo totale* salvato nell'applicazione;
- la *media* di entrate/uscite del mese corrente;
- i *filtri* per le transazioni;
- la *lista delle transazioni* ordinate dalla più recente;
- il *budget mensile* impostato;
- la sezione per *l'inserimento* e la *modifica* di nuove transazioni;
- due *grafici a torta* per le spese e entrate mensili divise per tag.

Tutte le funzionalità sono collegate tra di loro e si aggiornano in tempo reale in base ai dati presenti.

Le statistiche

Le prime informazioni subito accessibili sono il totale del portafoglio utente (dato dalla somma delle transazioni) e le medie mensili per quanto riguarda le spese e le entrate.

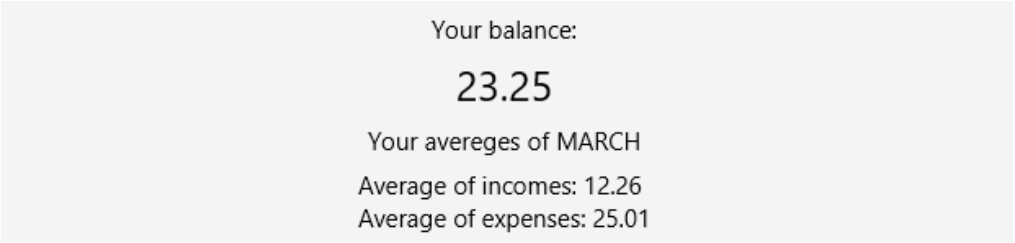

Figura A.1: Visualizzazione del totale e valore delle medie.

Le transazioni

Le transazioni sono il cuore dell'applicazione, grazie a queste tutte le altre funzionalità possono funzionare. Ogni transazione è caratterizzata da:

- un *tipo*; entrata (income) o uscita (expense);
- un *importo*;
- una *data*;
- una *descrizione testuale*;
- un *tag* che descrive la categoria alla quale la transazione appartiene.

Dopo aver aggiunto tutte le informazioni nell'apposito spazio dedicato, si può salvare la transazione, che verrà poi aggiunta alla lista sovrastante. La lista delle transazioni è *filtrabile* per tipo, data e tag ed è possibile, inoltre, *modificare* ed *eliminare* i passaggi di denaro.

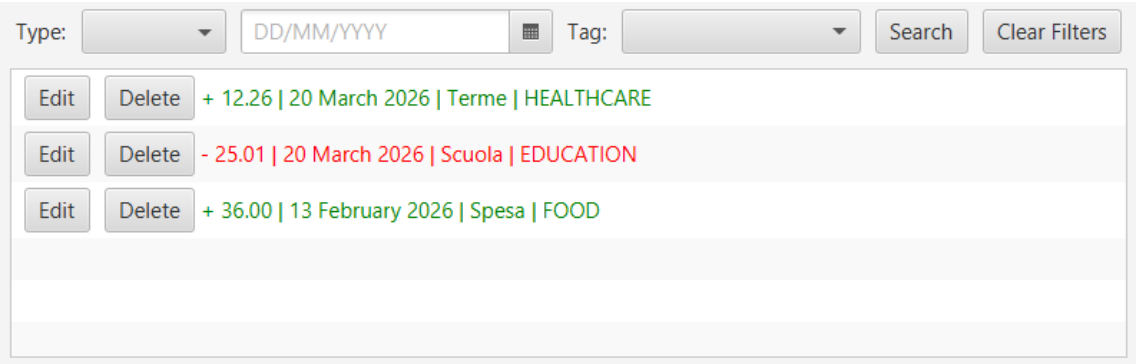

Figura A.2: Filtri per le transazioni e lista dei passaggi di denaro salvati su HeavyPocket.

Il budget

L'utente può impostare un *limite massimo di spesa mensile* tramite la sezione dedicata: questo serve per dare un'idea all'utente di quanto si è speso nel mese corrente, ma non blocca in alcun modo le transazioni.
Una volta impostato il budget, l'applicazione confronta automaticamente il totale speso nel mese corrente con il limite scelto: se il limite viene superato, viene segnalato l'accaduto.

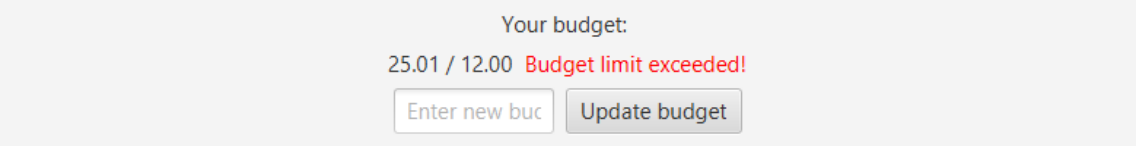

Figura A.3: Sezione per modificare e impostare un limite per il budget.

Gestione delle transazioni

Grazie ad appositi input form è possibile *aggiungere* delle transazioni, specificando per ogni campo il valore desiderato. Questo spazio è anche utilizzato per *modificare* direttamente i valori di una transazione già esistente.

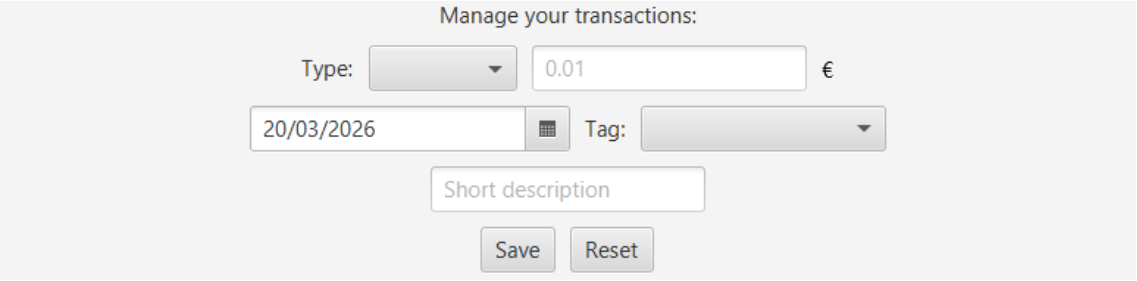

Figura A.4: Form input per aggiungere nuove transazioni e modificare quelle esistenti.

I grafici

L'applicazione infine permette all'utente di visualizzare velocemente, tramite grafici colorati, il proprio l'andamento finanziario. Questa sezione dell'applicazione non permette modifiche, poiché viene tutto calcolato automaticamente in base alle informazioni riportate all'interno della lista delle transazioni. In particolare, i movimenti di denaro vengono divisi per tipo e i loro importi sono cumulati per tag. Entrambi i *grafici a torta* mostrano quindi la presenza delle varie categorie nelle spese.

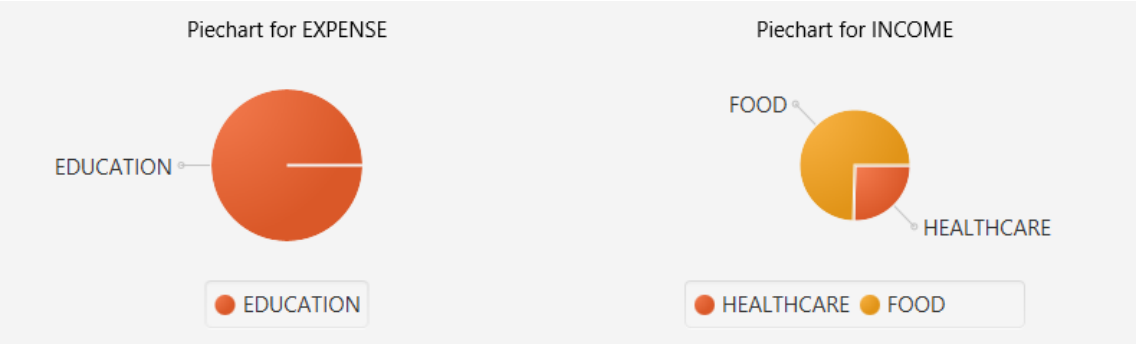

Figura A.5: Pannello che mostra i grafici di spese e entrate totali..

L'applicazione può essere utilizzata direttamente, senza l'utilizzo di dati importati, in quanto abbiamo gestito la persistenza e il funzionamento anche in mancanza degli stessi.

Appendice B

Esercitazioni di laboratorio

Purtroppo, essendo studenti al terzo anno, non abbiamo avuto la possibilità di seguire in presenza le lezioni in questo anno accademico a causa dello svolgimento del tirocinio, ergo non abbiamo potuto eseguire esercitazioni di laboratorio.

Bibliografia

- https://www.geeksforgeeks.org/system-design/mvc-design-pattern/#components-of-the-mvc-design-pattern
- https://italiancoders.it/observer-pattern/
- https://stackoverflow.com/questions
- https://www.baeldung.com/java-dto-pattern
- https://italiancoders.it/builder-pattern-un-ottima-alternativa-al-costruttore/
- https://docs.oracle.com/javase/8/javafx/api/javafx/scene/chart/PieChart.html
- https://docs.oracle.com/javafx/2/charts/pie-chart.htm
- https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html
- https://github.com/cric96/pss-pokedex-fx/tree/solution